



Experiment No. 14

File Processing In C

Objective:

The objectives of this lab are:

1. Understand the basic concepts of file processing in C programming.
2. Explain the role of files and streams in data storage.
3. Create and manipulate sequential-access files using standard file functions.
4. Read data from files using formatted and unformatted input functions.
5. Implement random-access files using structures and binary file operations.
6. Apply file positioning functions such as `fseek()` and `rewind()`.
7. Develop a transaction-processing program for updating file records.
8. Apply secure and efficient file-handling practices to prevent data loss and errors.

Prerequisites

Following are the prerequisites of this lab are:

- Understand basic C programming concepts such as variables, data types, and operators.
- Write and execute simple C programs using a compiler.
- Use conditional statements and loops in C.
- Understand the concept of functions in C.
- Have basic knowledge of arrays and structures.
- Be familiar with pointers and memory addressing.
- Understand basic input/output operations in C.
- Know how to use a C compiler and development environment.

Introduction

File processing in C enables programs to store and retrieve data permanently from secondary storage devices. Unlike variables, data stored in files is not lost when the program terminates. C provides a set of standard functions for creating, reading, writing, and updating files. File handling supports both sequential and random access methods for efficient data management. This experiment demonstrates practical implementation of file processing techniques in C.



Programming Fundamentals Lab Manual



Files and Streams:

In C, each file is viewed as a **sequential stream of bytes** that ends at an end-of-file marker. When a file is opened, a **stream** is associated with it to enable communication between the program and the file. At program start, three streams are automatically opened: **stdin**, **stdout**, and **stderr**, which handle keyboard input, screen output, and error messages. File operations are performed using a **FILE pointer**, and functions such as `fgetc`, `fputc`, `fgets`, `fputs`, `fscanf`, and `fprintf` are used to read from and write to files.

File Opening Modes:

Mode	Description
r	Open an existing file for reading.
w	Create a file for writing. If the file already exists, discard the current contents.
a	Append: open or create a file for writing at the end of the file.
r+	Open an existing file for update (reading and writing).
w+	Create a file for update. If the file already exists, discard the current contents.
a+	Append: open or create a file for update; writing is done at the end of the file.
rb	Open an existing file for reading in binary mode.
wb	Create a file for writing in binary mode. If the file already exists, discard the current contents.
ab	Append: open or create a file for writing at the end of the file in binary mode.
rb+	Open an existing file for update (reading and writing) in binary mode.
wb+	Create a file for update in binary mode. If the file already exists, discard the current contents.
ab+	Append: open or create a file for update in binary mode; writing is done at the end of the file.

Creating a Sequential-Access File

C files have no built-in structure, so we create our own records. Each record can store data like **account number**, **client name**, and **balance**. The account number can be used as a **key**, and records are usually entered in order. Sorting can be applied if needed before writing to the file.



Programming Fundamentals Lab Manual

Algorithm: Creating a Sequential-Access File:

1. Start
2. Declare a file pointer
3. Open file in write mode (w)
4. Write records using fprintf()
5. Close the file
6. Stop

Flowchart: Sequential File Creation:

Start → Open File (w) → Write Records → Close File → Stop

Program 1: Creating a Sequential-Access File

```
#include <stdio.h>

int main() {
    FILE *cfPtr; // File pointer
    int account;
    char name[30];
    double balance;
    int n, i;

    printf("Enter number of clients: ");
    scanf("%d", &n);

    // Open file in write mode
    cfPtr = fopen("clients.txt", "w");
    if (cfPtr == NULL) {
        printf("Error opening file.\n");
        return 1;
    }

    // Get client data and write to file
    for (i = 0; i < n; i++) {
        printf("Enter account number: ");
        scanf("%d", &account);
        printf("Enter client name: ");
        scanf("%s", name);
        printf("Enter balance: ");
        scanf("%lf", &balance);

        fprintf(cfPtr, "%d %s %.2f\n", account, name, balance);
    }
}
```



Programming Fundamentals Lab Manual



```
fclose(cfPtr); // Close the file
printf("Data successfully written to clients.txt\n");

return 0;
}
```

Algorithm: Reading a Sequential-Access File:

1. Start
2. Open file in read mode (r)
3. Read records using fscanf()
4. Display records
5. Close file
6. Stop

Program 2: Reading Data from a Sequential-Access File

```
#include <stdio.h>
int main(void) {
    FILE *cfPtr;
    int account;
    char name[30];
    double balance;
    cfPtr = fopen("clients.txt", "r");
    while (fscanf(cfPtr, "%d %s %lf", &account, name, &balance) != EOF)
    {
        printf("%d %-10s %.2f\n", account, name, balance);
    }
    fclose(cfPtr);
    return 0;
}
```

11.5 Random-Access Files



Programming Fundamentals Lab Manual

Random-access files store fixed-length records, which allows direct access to any record without reading the entire file. This makes them suitable for systems like banking, airline reservations, and point-of-sale, where fast access to specific data is required. Each record's location can be calculated from its record

key, enabling immediate reading, updating, or deletion. Fixed-length records ensure that inserting or modifying data does not affect other records, making random-access files efficient for transaction-processing applications. Random-access files allow direct access to any record using the `fseek()` function. Records are typically stored using structures and binary file operations.

Algorithm: Creating a Random-Access File:

1. Start
2. Define structure
3. Open file in binary write mode
4. Initialize records
5. Write records using `fwrite()`
6. Close file
7. Stop

Flowchart: Random-Access File

Start → Open File (wb) → Write Records → Close File → Stop

Program 3: Creating a Random-Access File:

```
#include <stdio.h>
#include <string.h>

struct clientData {
    int acctNum;
    char name[30];
    double balance;
};

int main() {
    FILE *cfPtr;
    struct clientData client;
    int n, i;

    printf("Enter number of clients: ");
    scanf("%d", &n);
```



Programming Fundamentals Lab Manual



```
// Open file in binary write mode
cfPtr = fopen("clients.dat", "wb");
if (cfPtr == NULL) {
    printf("Error opening file.\n");
    return 1;
}

// Enter client data and write to file
for (i = 0; i < n; i++) {
    printf("Enter account number: ");
    scanf("%d", &client.acctNum);
    printf("Enter client name: ");
    scanf("%s", client.name);
    printf("Enter balance: ");
    scanf("%lf", &client.balance);

    // Write record to file
    fwrite(&client, sizeof(struct clientData), 1, cfPtr);
}
fclose(cfPtr); // Close the file
printf("Random-access file created successfully.\n");

return 0;
}
```

Example Program: Reading a Random-Access File

```
#include <stdio.h>

struct clientData {
    int acctNum;
    char name[30];
    double balance;
};

int main() {
    FILE *cfPtr;
    struct clientData client;
    int acctNum;

    // Open file in binary read mode
    cfPtr = fopen("clients.dat", "rb");
```



Programming Fundamentals Lab Manual



```
if (cfPtr == NULL) {
    printf("Error opening file.\n");
    return 1;
}
printf("Enter account number to read: ");
scanf("%d", &acctNum);

// Move to record position
fseek(cfPtr, (acctNum - 1) * sizeof(struct clientData), SEEK_SET);
// Read the record
fread(&client, sizeof(struct clientData), 1, cfPtr);

printf("Account Number: %d\n", client.acctNum);
printf("Name: %s\n", client.name);
printf("Balance: %.2f\n", client.balance);
fclose(cfPtr);
return 0;
}
```

Explanation:

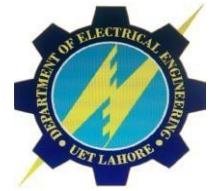
- Each record has a **fixed length** (defined by struct clientData), which allows **direct access** using fseek().
- fwrite() writes a record to the file; fread() reads a record.
- This method enables **fast access, update, or deletion** of any record without affecting other records.

Transaction-Processing Program:

A transaction-processing program uses a **random-access file** to manage client accounts efficiently. Each client record is stored with a **fixed-length structure**, allowing direct access using the account number as a key. The program can perform **deposit and withdrawal operations**, updating the client's balance without affecting other records. This approach ensures **fast, safe, and accurate processing** of transactions in systems like banking or point-of-sale.



Programming Fundamentals Lab Manual



Transaction-Processing Program: Random-Access File:

```
#include <stdio.h>
struct clientData {
    int acctNum;
    char lastName[15];
    char firstName[10];
    double balance;
};
int main(void) {
    FILE *cfPtr;
    struct clientData client;
    int choice;
    double amount;
    cfPtr = fopen("credit.dat", "rb+");

    printf("Enter account number (1-100): ");
    scanf("%d", &client.acctNum);

    fseek(cfPtr, (client.acctNum - 1) * sizeof(struct clientData), SEEK_SET);

    fread(&client, sizeof(struct clientData), 1, cfPtr);

    printf("1. Deposit 2. Withdraw: ");
    scanf("%d", &choice);
    printf("Enter amount: ");
    scanf("%lf", &amount);

    if (choice == 1)
        client.balance += amount;
    else if (choice == 2)
        client.balance -= amount;

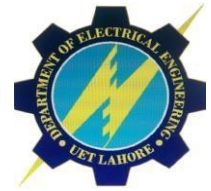
    fseek(cfPtr, (client.acctNum - 1) * sizeof(struct clientData), SEEK_SET);

    fwrite(&client, sizeof(struct clientData), 1, cfPtr);

    fclose(cfPtr);
    return 0;
}
```



Programming Fundamentals Lab Manual



Lab Tasks

Task 1

Create a sequential-access file to store **student records** (roll number, name, and marks). Write a program to **add, display, and search** records from the file.

Code

```
#include <stdio.h>
#include <string.h>

struct student {
    int roll;
    char name[50];
    double marks;
};

int main() {
    FILE *fp;
    int choice, search_roll, found;
    struct student s;

    while(1) {
        printf("\n1. add record\n2. display all\n3. search\n4. exit\nchoice: ");
        scanf("%d", &choice);

        if(choice == 1) {
            fp = fopen("students.txt", "a");
            // check if the file opened successfully
            if (fp == NULL) {
                printf("error: could not open file.\n");
                continue;
            }

            printf("enter roll, name, marks: ");
            scanf("%d %s %lf", &s.roll, s.name, &s.marks);
            fprintf(fp, "%d %s %.2f\n", s.roll, s.name, s.marks);
            fclose(fp);
        }
    }
}
```

```
} else if(choice == 2) {
    fp = fopen("students.txt", "r");
    if (fp == NULL) {
        printf("no records found (file might not exist yet).\n");
        continue;
    }

    printf("\n--- student records ---\n");
    // read until end of file
    while(fscanf(fp, "%d %s %lf", &s.roll, s.name, &s.marks) != EOF) {
        printf("roll: %d, name: %s, marks: %.2f\n", s.roll, s.name, s.marks);
    }
    fclose(fp);
} else if(choice == 3) {
    printf("enter roll to search: ");
    scanf("%d", &search_roll);

    fp = fopen("students.txt", "r");
    if (fp == NULL) {
        printf("no records found.\n");
        continue;
    }
}
```

```
found = 0;
while(fscanf(fp, "%d %s %lf", &s.roll, s.name, &s.marks) != EOF) {
    if(s.roll == search_roll) {
        printf("found -> name: %s, marks: %.2f\n", s.name, s.marks);
        found = 1;
        break;
    }
}
if(!found) printf("record not found.\n");
fclose(fp);

} else {
    break;
}
return 0;
}
```



Programming Fundamentals Lab Manual



Output

```
1. add record
2. display all
3. search
4. exit
choice: 1
enter roll, name, marks: 16
Umar
100

1. add record
2. display all
3. search
4. exit
choice: 2

--- student records ---
roll: 16, name: Umar, marks: 100.00

1. add record
2. display all
3. search
4. exit
choice: |
```



Programming Fundamentals Lab Manual



Task 2

Create a random-access file to store **product information** (product ID, name, price, quantity). Write a program to **insert, update, and display** product details at a specific record position using product ID as the key.

Code:

```
#include <stdio.h>

struct product {
    int id;
    char name[30];
    double price;
    int quantity;
};

int main() {
    FILE *fp;
    struct product p;
    int choice, target_id;

    fp = fopen("products.dat", "rb+");
    if (fp == NULL) {
        fp = fopen("products.dat", "wb+");
        if (fp == NULL) {
            printf("fatal error: could not create file. exiting.\n");
            return 1;
        }
    }

    while(1) {
        printf("\n1. insert/update product\n2. display product\n3. exit\nchoice: ");
        scanf("%d", &choice);
```

```
        if(choice == 1) {
            printf("enter product id (acts as memory slot): ");
            scanf("%d", &p.id);
            printf("enter name, price, quantity: ");
            scanf("%s %lf %d", p.name, &p.price, &p.quantity);

            fseek(fp, (p.id - 1) * sizeof(struct product), SEEK_SET);
            fwrite(&p, sizeof(struct product), 1, fp);
            printf("product saved.\n");

        } else if(choice == 2) {
            printf("enter product id to find: ");
            scanf("%d", &target_id);

            fseek(fp, (target_id - 1) * sizeof(struct product), SEEK_SET);
            fread(&p, sizeof(struct product), 1, fp);

            if(p.id == target_id) {
                printf("id: %d, name: %s, price: %.2f, qty: %d\n", p.id, p.name, p.price, p.quantity);
            } else {
                printf("no product exists at that id.\n");
            }

        } else {
            break;
        }
    }
    fclose(fp);
    return 0;
}
```

Output:

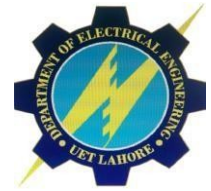
```
1. insert/update product
2. display product
3. exit
choice: 1
enter product id (acts as memory slot): 1
enter name, price, quantity: Panadol
30
1
product saved.

1. insert/update product
2. display product
3. exit
choice: 2
enter product id to find: 1
id: 1, name: Panadol, price: 30.00, qty: 1

1. insert/update product
2. display product
3. exit
choice: |
```



Programming Fundamentals Lab Manual



Task 3

Develop a program to **manage library transactions** using a random-access file. The program should allow **issuing and returning books**, update the book availability, and maintain records of borrowers.

Code:

```
#include <stdio.h>
#include <string.h>

struct book {
    int id;
    char title[50];
    int available;
    char borrower[30];
};

int main() {
    FILE *fp;
    struct book b;
    int choice, book_id;

    fp = fopen("library.dat", "rb+");
    if (fp == NULL) {
        fp = fopen("library.dat", "wb+");
        if (fp == NULL) {
            printf("fatal error: could not create file. exiting.\n");
            return 1;
        }
    }

    while(1) {
        printf("\n1. add book\n2. issue book\n3. return book\n4. display info\n5. exit\nchoice: ");
        scanf("%d", &choice);

        if(choice == 1) {
            printf("enter book id: ");
            scanf("%d", &b.id);
            printf("enter title: ");
            scanf("%s", b.title);

            b.available = 1;
            strcpy(b.borrower, "none");
        }
    }
}
```

```
fseek(fp, (b.id - 1) * sizeof(struct book), SEEK_SET);
fwrite(&b, sizeof(struct book), 1, fp);
printf("book added to library.\n");

} else if(choice == 2 || choice == 3) {
    printf("enter book id: ");
    scanf("%d", &book_id);

    fseek(fp, (book_id - 1) * sizeof(struct book), SEEK_SET);
    fread(&b, sizeof(struct book), 1, fp);

    if(b.id == book_id) {
        if(choice == 2 && b.available == 1) {
            printf("enter borrower name: ");
            scanf("%s", b.borrower);
            b.available = 0;
            printf("book issued to %s.\n", b.borrower);
        } else if (choice == 3 && b.available == 0) {
            b.available = 1;
            strcpy(b.borrower, "none");
            printf("book returned successfully.\n");
        } else {
            printf("action failed. check if already issued/returned.\n");
        }

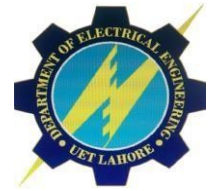
        fseek(fp, (book_id - 1) * sizeof(struct book), SEEK_SET);
        fwrite(&b, sizeof(struct book), 1, fp);
    } else {
        printf("book not found.\n");
    }
} else if(choice == 4) {
    printf("enter book id: ");
    scanf("%d", &book_id);

    fseek(fp, (book_id - 1) * sizeof(struct book), SEEK_SET);
    fread(&b, sizeof(struct book), 1, fp);
```

```
    if(b.id == book_id) {
        printf("title: %s, available: %s, borrower: %s\n",
            b.title, b.available ? "yes" : "no", b.borrower);
    } else {
        printf("book not found.\n");
    }
} else {
    break;
}
}
fclose(fp);
return 0;
```



Programming Fundamentals Lab Manual



Output:

```
umar@umarr:~/Downloads/PF-Labs-main/Lab14_2025
```

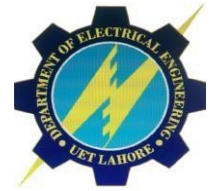
```
1. add book
2. issue book
3. return book
4. display info
5. exit
choice: 1
enter book id: 1
enter title: CAD
book added to library.
```

```
1. add book
2. issue book
3. return book
4. display info
5. exit
choice: 4
enter book id: 1
title: CAD, available: yes, borrower: none
```

```
1. add book
2. issue book
3. return book
4. display info
5. exit
choice:
```



Programming Fundamentals Lab Manual



Bugged Program: Writing and Reading a Sequential-Access File:

```
#include <stdio.h>

int main() {
    FILE *fp;
    int roll;
    char name[20];
    double marks;
    int n, i;

    printf("Enter number of students: ");
    scanf("%d", &n);

    // BUG: file opened in read mode instead of write mode
    fp = fopen("students.txt", "r");
    if (fp == NULL) {
        printf("Cannot open file.\n");
        return 1;
    }

    for (i = 0; i <= n; i++) {    // BUG: should be i < n
        printf("Enter roll number: ");
        scanf("%d", &roll);
        printf("Enter name: ");
        scanf("%s", name);
        printf("Enter marks: ");
        scanf("%lf", &marks);

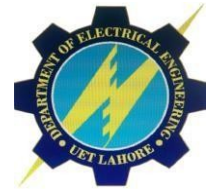
        // BUG: using fscanf instead of fprintf to write
        fscanf(fp, "%d %s %lf\n", &roll, name, &marks);
    }

    fclose(fp);

    printf("Data written successfully.\n");
    return 0;
}
```



Programming Fundamentals Lab Manual



Corrected Code

```
#include <stdio.h>

int main() {
    FILE *fp;
    int roll;
    char name[20];
    double marks;
    int n, i;

    printf("Enter number of students: ");
    scanf("%d", &n);

    fp = fopen("students.txt", "w");
    if (fp == NULL) {
        printf("Cannot open file.\n");
        return 1;
    }

    for (i = 0; i < n; i++) {
        printf("Enter roll number: ");
        scanf("%d", &roll);
        printf("Enter name: ");
        scanf("%s", name);
        printf("Enter marks: ");
        scanf("%lf", &marks);

        fprintf(fp, "%d %s %.2f\n", roll, name, marks);
    }

    fclose(fp);
    printf("Data written successfully.\n");
    return 0;
}
```

Output:

```
umar@umarr:~/Downloads/PF-Labs-main/Lab14_2025_EE_016$ ./task4
Enter number of students: 2
Enter roll number: 1
Enter name: Umar
Enter marks: 100
Enter roll number: 2
Enter name: Haseeb
Enter marks: 100
Data written successfully.
umar@umarr:~/Downloads/PF-Labs-main/Lab14_2025_EE_016$ |
```



Programming Fundamentals Lab Manual

